Academic Year -: 2026-27

Date: 11-07-2026




SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

B-Tech Project

# ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

**Under the guidance of**

Dr. Payal Kadam

| Group Number | **4** | |
|---|---|---|
| Name | PRN | CLASS |
| Faheemuddin Sayyed | 23070122196 | CSE - C |
| Sanidhya Awasthi | 23070122192 | CSE - C |
| Raghav Sonchhatra | 23070122172 | CSE - C |

## PROBLEM STATEMENT

ForgeFlow - an AI-native, human-in-the-loop web platform that carries a product idea from screenshots or a structured spec through automated UI design, full-stack code generation, requirement-driven testing with self-healing repair, and one-click cloud deployment, while letting the user refine, revisit, or skip any stage at will.

## ABSTRACT

Modern AI application builders such as Lovable, Bolt, and Replit Agent have collapsed the distance between an idea and a running web application, yet they share three weaknesses: code generation is loosely verified, so users are frequently trapped in unbounded bug-fixing loops; testing of the live, deployed application is rarely a first-class step; and deployment either locks users into a single proprietary cloud or exposes them to infrastructure configuration that non-experts cannot navigate. ForgeFlow is an AI-native, human-in-the-loop platform that automates the full software development lifecycle for web applications while keeping the developer in control at every stage.

Delivered as a Progressive Web App with an embedded browser IDE, the platform accepts an idea expressed as screenshots, a natural-language description, or the user's own designs, and moves it through five orchestrated stages. First, design: reference images or prompts are converted into high-fidelity UI through a pluggable provider (Google Stitch over the Model Context Protocol, with Figma as an alternative). Second, generation of a fixed, reliable stack - React frontend, Node/Express backend, and MongoDB - driven by an agentic loop over a large language model, with code streamed live to the IDE. Third, verification: a structured, fill-in-the-blanks requirements guide is compiled into executable unit and end-to-end (Playwright) tests. Fourth, infrastructure-aware deployment, offering either seamless provisioning on platform-managed accounts (Vercel and Render) or the user's own credentials, rendered as a live component topology. Fifth, validation of the deployed site against the same tests.

The core contribution is a bounded, diff-aware self-healing loop: when tests fail, only the failing output and the relevant code diff are returned to the agent for targeted repair, governed by retry caps and no-progress detection that escalate to the human rather than looping indefinitely. Crucially, no stage is mandatory - users can refine any output conversationally, re-enter earlier stages, or opt out entirely, making the pipeline fully non-linear. Each project is provisioned an isolated database with a built-in CRUD data browser.

The system is evaluated on a benchmark of representative application specifications, measuring first-pass and post-repair test success, repair-iteration counts, and end-to-end time from screenshot to deployed URL - demonstrating that disciplined verification and flexible human control can coexist with fully automated build-and-deploy.

**Outcomes**

1. A deployed PWA where a user goes from a few reference screenshots to a live, hosted website URL end to end.

2. Image/prompt → high-fidelity UI generation through a pluggable design layer (Stitch MCP default, Figma MCP alternative).

3. Agentic full-stack generation on a fixed stack (React + Node/Express + MongoDB) with code streamed live into an in-browser Monaco IDE.

4. Isolated, per-project execution sandbox with live preview of the running app.

5. A structured, fill-in-the-blanks requirements capture that compiles directly into unit and Playwright E2E test suites.

6. A bounded, diff-aware self-healing repair loop with retry caps, no-progress detection, and human escalation - the project's central technical contribution.

7. Cloud-infra analysis that classifies each project's needs and recommends the simplest viable deployment target.

8. Dual-mode deployment: seamless on platform-managed accounts, or bring-your-own tokens/credentials.

9. Post-deployment validation that re-runs E2E/smoke tests against the live URL and repairs on failure.

10. An n8n-style visual deployment topology showing FE/BE/DB components with live status.

11. Per-project database isolation plus a project-scoped CRUD data browser; optional user-supplied MongoDB URI.

12. Fully non-linear orchestration: every stage is optional, conversationally refinable, and re-enterable.

13. An evaluation harness reporting first-pass vs post-repair test success, repair-iteration counts, and screenshot-to-URL time - the basis for a paper or a hackathon pitch.

# User Flow Diagram

